DigitForge - Project Explanation

DigitForge is a local Streamlit app. You upload an image of handwritten digits; it finds each digit, converts it to a 28x28 MNIST-style image, reconstructs it with two separately trained generative models (an **RBM** and a **VAE**), compares them, and shows measured performance graphs. Everything runs on your machine with PyTorch (GPU if CUDA is available, otherwise CPU). There is no backend server.

1. How the app flows (Home page)

• **Upload** an image (or use the camera) - digits are detected automatically.

• **Reconstruct** - each digit is rebuilt independently by the RBM and the VAE; MSE and MAE are shown against the preprocessed digit; images can be downloaded.

• **Model comparison** - original beside both reconstructions, plus a per-metric ranking from the actual values.

• **Graphs** - five measured graphs on the 10,000 held-out MNIST test images.

• **About the Models** page - short theory of the RBM and VAE.

On first launch, any model without a checkpoint is trained automatically with fixed settings (RBM: 512 hidden units, CD-1, 25 epochs; VAE: latent size 20, 20 epochs; batch 128; all 60,000 MNIST images; seed 42). Later launches reuse the saved checkpoints.

2. Modules

File	What it does
app.py	Entry point. Builds the Home page (upload, reconstruct, compare, graphs) and the About navigation; triggers first-run training.
sections/reconstruction.py	Upload/camera input, detection preview, controls (RBM Gibbs steps, seed), reconstruction, MSE/MAE table, downloads, VAE latent interpolation.
sections/comparison.py	Shows original + RBM + VAE side by side and ranks the models per metric using the correct direction (lower or higher is better).
sections/analytics.py	Runs the held-out evaluation once (cached) and draws the five graphs with captions computed from measured numbers.
pages/About.py	Explains the RBM, VAE and the evaluation method.
models/rbm.py	Bernoulli-Bernoulli RBM: sampling, free energy, Contrastive Divergence update, Gibbs-sampling reconstruction.
models/vae.py	Convolutional VAE: encoder (mean, log-variance), reparameterization, decoder, ELBO loss, reconstruction and latent interpolation.
training/configs.py	Fixed training settings (dataclasses) for both models.
training/common.py	MNIST loading (train/test), seeded batching, stop handling, command-line helpers.
training/train_rbm.py	Trains the RBM with CD-k; logs reconstruction error and train/validation free energy; saves the checkpoint.
training/train_vae.py	Trains the VAE; logs total, reconstruction and KL losses (train and validation); saves the checkpoint.
utils/preprocessing.py	Image loading (EXIF fix, transparency to white), grayscale, background-polarity detection, Otsu thresholding, 28x28 normalization.
utils/digit_detection.py	Connected components, merging broken pieces, speck removal, ordering, crops and preview boxes.
utils/reconstruction.py	Loads checkpoints into an engine and reconstructs digits with each model, then scores them.
utils/metrics.py	MSE, MAE, PSNR, SSIM and metric ranking.

File	What it does
utils/evaluation.py	Held-out test evaluation, latency benchmark, parameter and checkpoint-size measurement; caches results in outputs/.
utils/plotting.py	Matplotlib figures and digit-image helpers (fixed colour per model).
utils/checkpoints.py	Safe save/load with a format version and stored config; clear errors for missing or incompatible files.
utils/ui.py, device.py, paths.py	Cached engine loading, first-run training UI, CPU/GPU selection, project folders.
tests/	26 pytest tests: preprocessing, detection, shapes, metrics, checkpoints, tiny end-to-end training.

3. Preprocessing and digit extraction

• Grayscale conversion; the background is judged from the image border versus the Otsu threshold, so both dark-on-light and light-on-dark images end up as white digit on black (MNIST convention).

• Otsu threshold, then connected components. Pieces stacked above each other are merged (e.g. a broken 5); tiny specks are dropped; digits are ordered left to right (row by row for several lines).

• Each digit: tight crop, resized to fit 20x20 **keeping its aspect ratio**, thin strokes thickened to MNIST width, centred on a 28x28 canvas by centre of mass, scaled to [0,1].

4. The two models

RBM - an energy-based model with binary visible and hidden units, trained by Contrastive Divergence. To reconstruct, the digit is binarized and clamped on the visible units, hidden units are inferred, then a configurable number of Gibbs steps (hidden to visible to hidden ...) is run; the output is the final visible probability map. More steps drift further from the input.

VAE - the encoder maps the digit to a mean and variance of a latent Gaussian; the reparameterization trick (z = mean + sigma * noise) allows gradient training; the decoder rebuilds the image. The loss is reconstruction (binary cross-entropy) plus KL divergence. In the app the VAE always decodes the latent **mean** (deterministic). The interpolation feature decodes points between the latent means of two digits.

5. Metrics

• **MSE** - mean squared pixel error (lower is better). **MAE** - mean absolute pixel error (lower is better).

• PSNR and SSIM are still computed internally and used in the comparison ranking, but only MSE and MAE are shown in the upload table and Graph 1.

• For an uploaded image there is no clean ground truth: metrics compare against the preprocessed digit, so they measure similarity, not whether the digit is still the same digit.

6. The graphs (10,000 held-out MNIST test images, RBM with 1 Gibbs step)

Graph	What it shows	How to read it / measured result
Graph 1 - Reconstruction quality	Bar chart of mean MSE and MAE for RBM and VAE (whiskers = standard deviation).	Lower is better. Measured: MSE RBM 0.0106 vs VAE 0.0092 (VAE better); MAE RBM 0.0303 vs VAE 0.0323 (RBM better). Different metrics favour different models, so no single overall winner.
Graph 2 - Training convergence	RBM reconstruction MSE per epoch; VAE total loss, reconstruction loss and KL per epoch (log scale).	Shows whether training settled. The objectives differ per model, so absolute values are not comparable across panels.

Graph	What it shows	How to read it / measured result
Graph 3 - Inference time	Mean milliseconds per image (batch of 1, warm-up discarded, 100 runs, device synchronised, model loading excluded).	Measured: RBM about 0.27 ms, VAE about 0.52 ms on this machine. Both are far below what a user can notice.
Graph 4 - Error distribution	Histogram of per-image MSE over all test images; dotted lines are the medians.	Median MSE: RBM 0.0105, VAE 0.0080. The VAE is typically better; the 95th percentile (RBM 0.0162, VAE 0.0199) shows the VAE has a slightly longer tail of badly rebuilt digits.
Graph 5 - Model complexity	Parameters used at inference, checkpoint size and measured latency.	RBM about 0.40 M parameters / 3.1 MB; VAE about 1.69 M parameters / 6.5 MB. The VAE is larger and slower per image.

These numbers come from the most recent evaluation run saved in outputs/evaluation.json. They change if you retrain or change the RBM Gibbs steps; the app recomputes them automatically.

7. Limitations

• Reconstruction does not guarantee the same digit; metrics measure pixel similarity only.

• Digits that touch each other form one connected shape and are not split automatically.

• The RBM binarizes the input at 0.5, so anti-aliasing detail is lost for that model only.

• First-run training blocks the page until it finishes (minutes on GPU, longer on CPU).